Evaluating Cryptographic API Misuse Detectors for Go

Vivi Andersson, Martin Monperrus

KTH Royal Institute of Technology, Stockholm, Sweden

April 18, 2026
Rio de Janeiro, Brazil
SVM 2026

A Cryptographic API Misuse

Traefik, CVE-2025-66491. ingress-nginx provider:

```
InsecureSkipVerify: strings.ToLower(
    ptr.Deref(cfg.ProxySSLVerify, "off")) ==
        "on",
```

- The user sets `proxy-ssl-verify: "on"`, expecting TLS verification to be **enabled**.
- Expression evaluates to `true`, `InsecureSkipVerify` becomes `true`, inverting security assumptions.
- TLS encryption remains, but backend authentication is disabled: Traefik may trust a spoofed backend.

Why Go

- **Go runs critical infrastructure:** Kubernetes, Terraform, Traefik, Let's Encrypt's Boulder.
- Cryptographic misuses reach production, for example CVE-2025-66491 in Traefik and CVE-2024-45337 in `golang.org/x/ssh`.
- Four Go detectors exist (GOPHER, CODEQL, GOSEC, SNYK CODE), but none have been systematically evaluated.

Contribution

The first comparative study of cryptographic API misuse detectors for Go.

Research Questions

- **RQ1.** What is the design space of state-of-the-art tools for detecting cryptographic API misuses in Go?
  - We survey the tools and consolidate a taxonomy of 14 Go crypto misuse classes.
- **RQ2.** To what extent are crypto misuse detection tools for Go consistent with each other?
  - We run all four tools on 328 security-relevant Go projects and compare their findings.

Sourced Tools

CodeQL

GitHub • Open source

Datalog queries over a fact database, with data-flow reasoning.

Gosec

Community • Open source

SSA for some rules, AST pattern matching for others.

Gopher

Academic • Binary-only

SSA-based taint tracking, with dynamically updatable rules.

Snyk Code

Commercial • Proprietary

Static analysis product; implementation details undisclosed.

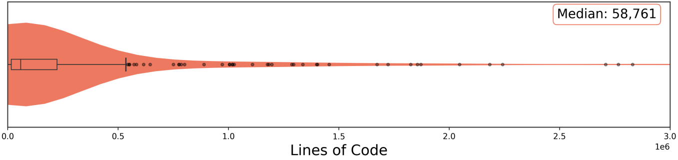
RQ1: Taxonomy and Tool Coverage

Category	#	Rule	CodeQL	Gopher	Gosec	Snyk
Primitives	01	Insecure algorithms		✓	✓	✓
	02	Insecure PRNG	✓	✓	✓	✓
	03	Deprecated Go function		✓		
Key mgmt	04	Constant/predictable key		✓		
	05	Short key length	✓	✓	✓	✓
	06	Static/predictable IV		✓	✓	
PBKDF	07	Short salt length		✓		
	08	Predictable salt		✓		
	09	Low hash iterations		✓		
Transport	10	HTTP protocol		✓		
	11	TLS/SSL issues	✓	✓	✓	✓
SSH	12	Insecure SSH suite		✓		
	13	No host key validation	✓	✓	✓	
Token	14	No JWT verification	✓			

- **Coverage:** Gopher (13) > Gosec (6) > CodeQL (5) > Snyk Code (4)
- **All tools:** 3 of 14 rules (#02, #05, #11) are covered by all

RQ2: Dataset

- **328 open-source Go repositories**, filtered from a curated set of 920 popular projects.
- Kept only actively maintained projects (commit within the past year, stable release) where cryptography is core.
- Size is right-skewed: median 58,761 LOC.



RQ2: Tool Execution and Coverage

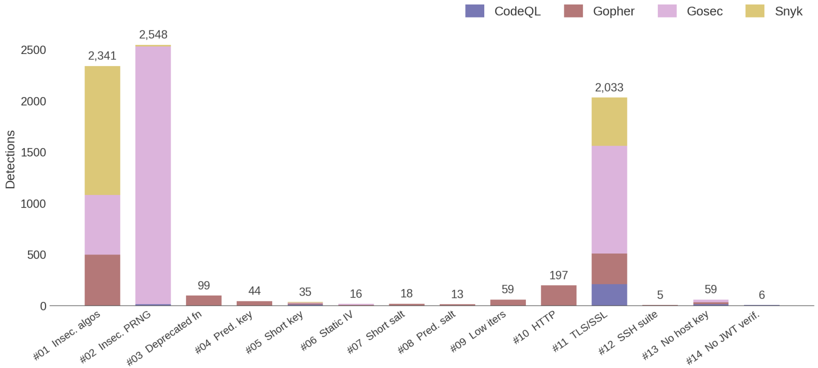
	CodeQL	Gopher	Gosec	Snyk
Projects analyzed (%)	91.8	77.1	100	100
With findings (% of analyzed)	30.9	64.8	84.5	92.7
With findings (% of dataset)	28.4	50.0	84.5	92.7

Coverage

Snyk Code and Gosec analyze all 328 projects, while Gopher fails on 75 (runtime errors) and CodeQL on 27 (database build erros).

RQ2: Raw Findings by Tool

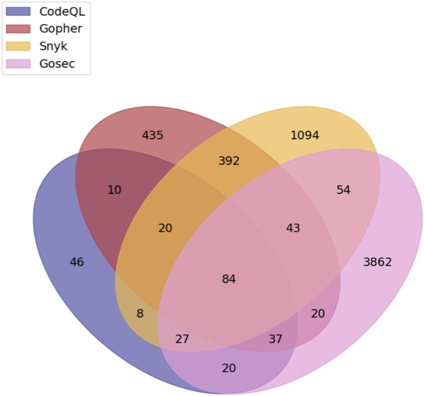
- 7,473 findings across 328 projects



- Gopher uniquely detects issues in #04 Predictable Key
- Excessive detections indicate insensitive detection (e.g., #01, #02, #11)

RQ2: Line Level Agreement

- 6,152 unique flagged lines across the dataset.
- Only **1.3%** are flagged by **all four** tools. **11.6%** are flagged by at least **two**.



RQ2: Per-Rule Agreement

Category	#	Rule	CodeQL	Gopher	Gosec	Snyk
Primitives	01	Insecure algorithms		✓	✓	✓
	02	Insecure PRNG	✓	✓	✓	✓
	03	Deprecated Go function		✓		
Key mgmt	04	Constant/predictable key		✓		
	05	**Short key length**	✓	✓	✓	✓
	06	Static/predictable IV		✓	✓	
PBKDF	07	Short salt length		✓		
	08	Predictable salt		✓		
	09	Low hash iterations		✓		
Transport	10	HTTP protocol		✓		
	11	**TLS/SSL issues**	✓	✓	✓	✓
SSH	12	Insecure SSH suite		✓		
	13	**No host key validation**	✓	✓	✓	
Token	14	No JWT verification	✓			

- Per-rule agreement matches on (`project, file, line, rule`)

RQ2: Agreement on Rule #05

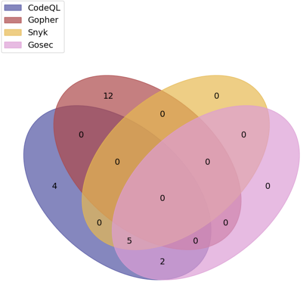
WIP

Concrete misuse: short RSA key

```
priv, err := rsa.GenerateKey(
    rand.Reader, 1024)
```

- 1024-bit RSA is below current recommendations.
- Weak keys make long-term impersonation or decryption more realistic.

Overlap. All four tools support Rule #05, but agreement is still low: they differ on whether mock/example code and broader patterns should count.



RQ2: Agreement on Rule #11

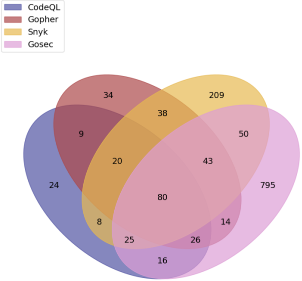
WIP

Concrete misuse: TLS certificate checks disabled

```
tls.Config{
    InsecureSkipVerify: true,
}
```

- TLS stays encrypted, but peer authentication is gone.
- A machine-in-the-middle can present its own certificate and be trusted.

Overlap. This is the largest class. All tools find true positives here, but GOSEC and SNYK CODE contribute many tool-unique alerts, with different precision levels.



RQ2: Agreement on Rule #13

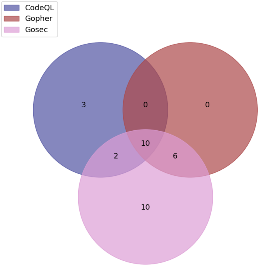
WIP

Concrete misuse: SSH host key verification disabled in `keadm batch`

```
HostKeyCallback: func(...) error
    {
    return nil
},
```

- The SSH client accepts any server host key, so the remote node identity is never authenticated.
- If an attacker redirects the connection, `keadm batch` may connect to the wrong host; with password authentication, SSH credentials may be exposed.

Overlap. GOSEC primarily matches `InsecureIgnoreHostKey`. CODEQL also detects equivalent custom callbacks, so the low overlap reflects deeper semantic coverage.



Issues in Practice

WIP, maybe covered in the examples.?

1. Context-dependent usage *Rule #05, flagged by* GOPHER

```
h := blake2b.New512(nil)   // bug if used as a MAC
                           // fine for content hashing
```

Static analysis cannot distinguish security-critical vs. benign usage.

2. Pattern match misses the equivalent *Rule #13, only* CODEQL

```
cfg.HostKeyCallback = func(...) error { return nil }
```

CODEQL detects this pattern; the other tools do not report it in our study.

3. Field name flagged, value ignored *Rule #11,* GOSEC *false positive*

```
tlsConfig := &tls.Config{Certificates: []tls.Certificate{cert}, RootCAs: roots}
setTLSDefaults(tlsConfig) // MinVersion 1.2 = OK
```

GOSEC lacks data flow capabilities and flags false positives.

Implications

- The four detectors barely overlap. Only 1.3% of flagged lines are shared by all four, and 92.6% of GOSEC's findings are unique to it.
- Tools that "support" the same rule often define misuse differently. GOSEC flags 2,517 insecure PRNG cases where GOPHER flags zero, on the same code.
- For now, no single tool is reliable on its own. Running multiple detectors and treating overlap as higher-confidence is the practical strategy.

Open Questions

- **Standardization of misuse**. Tools currently disagree because each defines rules its own way. A shared specification would give detectors a common target.
- **Context-sensitive analysis**. Several disagreements come from limited context sensitivity around how values are configured and used.

Questions

Thank you

Questions and discussion